PersistHotStuff: Designing a Practical Byzantine Fault-Tolerant Consensus

Saanvi Shet[1], Tejas Kolhe[1], and Vahida Attar[1]

Department of Computer Engineering, COEP Technological University, Pune, India
{shetsr22.comp, kolhetv22.comp, vahida.comp}@coeptech.ac.in

Abstract. Building a working Byzantine Fault-Tolerant (BFT) consensus system is harder than the theory suggests. HotStuff [1] offers an elegant $O(n)$ protocol with a clean 3-chain commit rule, yet every prototype we studied; and the Stanford CS244B report [7] confirms this; shares the same blind spots: no crash-safe storage, fixed validator sets, a toy state machine, and a pacemaker that stalls when clients go quiet. We set out to close these gaps. *PersistHotStuff* is our Rust-based framework built around four proposed extensions to HotStuff: a WAL-and-snapshot persistence layer, consensus-driven dynamic membership, a gRPC-backed pluggable state machine interface, and a pacemaker that injects dummy blocks to keep the 3-chain moving. We provide a TLA+ specification of the dynamic membership protocol, verified by TLC model checking across 7.3 million distinct states with zero violations of 11 safety invariants, and benchmark the system against analytical models of PBFT and Raft, confirming $O(n)$ message complexity with 9× fewer messages than PBFT at n=13.

Keywords: Byzantine Fault Tolerance, HotStuff, Distributed Consensus, Persistence, Dynamic Membership, TLA+, Formal Verification, Rust

1 Introduction

Anyone who has tried to turn a BFT consensus paper into running code knows the frustration: the protocol itself may be clean, but everything around it, like storage, configuration changes, application hooks and keeping the system alive when nobody is sending requests, gets hand-waved away. HotStuff [1] is a good example. On paper it is one of the tidiest BFT protocols available: $O(n)$ messages per decision, optimistic responsiveness, and a 3-chain commit rule that is easy to reason about. Meta trusted it enough to use it as the consensus engine for Diem. But what about the publicly available prototypes? They crash and lose everything. They assume that the validator set never changes. They hard-code a trivial state machine, and if clients stop sending requests for a while, the protocol stalls.

We are not the first to notice. A recent Stanford CS244B course report [7] catalogued exactly these four pain points after the authors tried to build a working HotStuff system from scratch.

1. **No persistence.** State lives in RAM. Kill a replica and it forgets everything, including but not limited to committed blocks, votes, and quorum.
2. **Static membership.** The validator set is baked in at startup. If a fifth node needs to be added, we would need to restart the whole cluster.
3. **Hard-coded state machine.** The "application" is usually an append-only log with no hooks for real business logic.
4. **Liveness stalls under low load.** The 3-chain rule needs three consecutive justified blocks. No client requests means no new blocks, which means nothing ever commits.

We started this project because we wanted a HotStuff implementation that addresses all four issues in one coherent system. The result is *PersistHotStuff*, whose design proposal we will present here, backed by implementation in Rust[13].

Our contributions, then, are:

- A **Persistence layer** made up of write-ahead log (WAL) + snapshots adapted from Raft [3] to the Byzantine setting.
- A **feature for dynamic membership** via consensus-driven reconfiguration commands.
- A **pluggable state machine interface** (trait-based, with gRPC for out-of-process applications).
- A **pacemaker design** that uses dummy proposals to keep the 3-chain growing during idle periods.
- An **open-source Rust codebase** with Ed25519 authentication, a simulation framework, and preliminary test results.

2 Background

2.1 System Model

We assume the standard partially synchronous model of Dwork et al. [10]: n replicas, up to f of which may be Byzantine (arbitrary behaviour, including collusion), with $n \geq 3f+1$. Messages are eventually delivered once the network stabilises, but there are no guarantees during asynchronous periods. This model is the baseline for most modern BFT work, including HotStuff, so we adopt it without modification.

Cluster-size restriction. While the general BFT lower bound is $n \geq 3f+1$, PersistHotStuff additionally requires $n \equiv 1 \pmod 3$, yielding valid cluster sizes $n \in \{4, 7, 10, 13, \ldots\}$. For any such n the fault threshold is exactly $f = (n-1)/3$, so the condition collapses to a tight equality $n = 3f+1$ rather than a lower bound.

2.2 How HotStuff Works

We briefly recap the mechanics since our design builds directly on them. HotStuff [1] runs in *views*. Each view has a leader who does three things: proposes

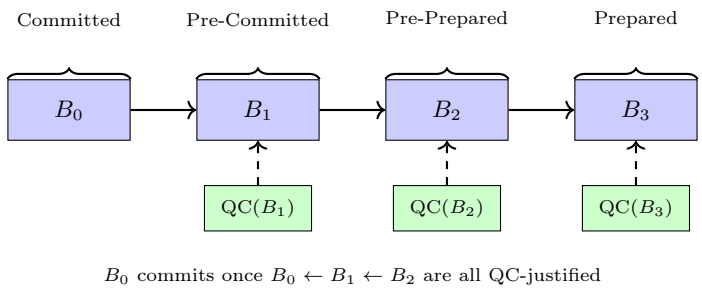

Fig. 1. The 3-chain commit rule. B_0 is only safe to commit once two successive QC-justified blocks extend it.

a block which contains the command that needs to be executed, collects at least $2f+1$ signed votes into a Quorum Certificate(QC), and broadcasts the QC. Safety comes from the *3-chain commit rule*: a block B_0 is considered committed only when there is a chain $B_0 \leftarrow B_1 \leftarrow B_2$ in which both B_1 and B_2 carry valid QC's. Until that chain forms, B_0 is not executed.

The protocol's appeal is its simplicity. Every decision costs $O(n)$ messages (one broadcast from the leader, $n-1$ votes back, one QC broadcast). If the leader is honest and the network is behaving, the system moves at actual network speed rather than waiting for timeouts. Fig. 1 illustrates the 3-chain structure.

3 Related Work

It helps to understand what each related system does well and where it stops short, since our design borrows selectively from several of them.

Practical Byzantine Fault Tolerance(PBFT) [2] proved that practical BFT was possible, but its $O(n^2)$ all-to-all communication makes it painful beyond a couple of dozen nodes. HotStuff [1] brought that down to $O(n)$ by funnelling everything through a leader and using QC's. More recent variants such as HotStuff-1 [4] tries to cut one communication phase, and Sync HotStuff [5] gets better throughput when the network is synchronous by focusing on squeezing out better latency or throughput from the core protocol. None of them tackle the four practical gaps we are interested in.

For persistence, the closest analogue is Raft [3]. Its WAL-plus-snapshot approach has been tested in systems such as etcd, and the recovery semantics are well understood. The catch is that Raft assumes only crash faults; transplanting its storage patterns into a Byzantine setting requires some care. For instance, a recovering replica cannot just trust its own log, it needs checksum verification and must re-validate against the quorum.

DynaNet [6] is one of the few BFT systems that explicitly supports dynamic validator sets. Our membership module takes a simpler route: we treat `JOIN` and `REMOVE` as regular consensus commands so that the existing 3-chain commit rule provides the atomicity guarantee for free.

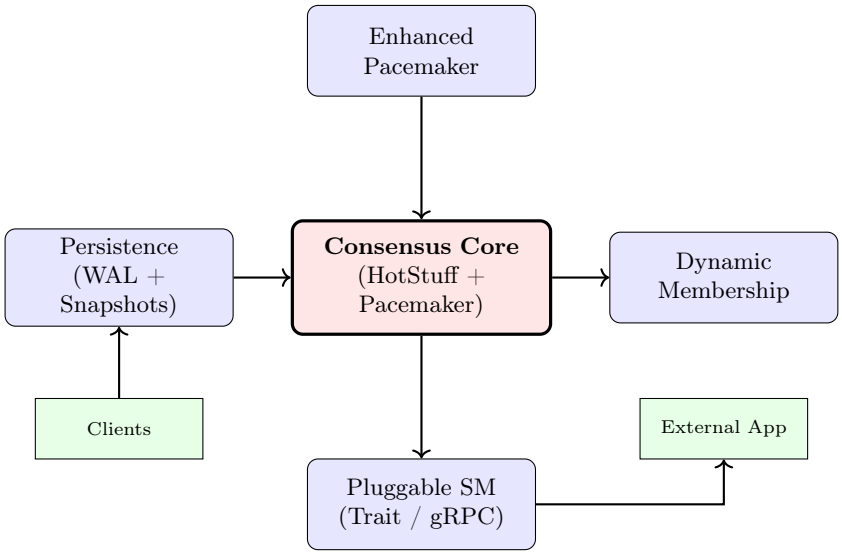

Fig. 2. High-level architecture. Solid arrows show data flow

Tendermint [8] deserves credit for popularising the idea of separating consensus from application logic through its ABCI. However, its main focus is on decentralised apps(dApps). Our `App` trait and gRPC interface borrow this philosophy directly and apply it to applications in general. We have targeted a Rust-native trait as the primary interface, with gRPC as an optional bridge for applications written in other languages.

As for liveness, the idea of injecting no-op commands when the system is idle goes back to early Paxos literature. We are not claiming novelty there, just noting that nobody seems to have wired it into a HotStuff pacemaker properly.

4 System Design

Our design is modular by necessity. Each of the four issues touches a different part of the system, so we treat them as separate modules that plug into a shared consensus core. Fig. 2 shows how the pieces fit together.

4.1 Persistence Layer (WAL + Snapshots)

The idea is to write every state-changing event to an append-only log *before* updating the in-memory state, and periodically take a full snapshot so the log can be truncated. What makes the Byzantine setting different is that a recovering replica cannot blindly trust what it reads off disk. A corrupted disk block or a Byzantine operator who tampered with the log file must be detected, not silently absorbed. So every WAL entry carries a CRC32 checksum, and the file header includes a SHA-256 digest of its own fields.

Offset	Size	Field	Description
0	4	Magic Number	0x48534C57 (“HSLW”)
4	2	Version	Format version (currently 0x0001)
6	8	Replica ID	Owner of this log
14	8	Created Time	Unix timestamp (ms)
22	32	Header Checksum	SHA-256 over the preceding fields

Table 1. WAL file header layout

Entry Frame		
Length (4 B)	CRC32 (4 B)	Payload (variable)

Table 2. Each WAL entry is length-prefixed and checksummed.

Tables 1 and 2 show the binary layout. We keep it simple; there is no need for a complex log-structured storage engine here; since the WAL is strictly append-only and reads happen only during recovery. All sizes mentioned in 1 are measured in bytes.

Every entry is `fsync`'d before the corresponding in-memory update proceeds. This means a crash at any point leaves the log in a recoverable state: either the entry was fully written (and its checksum will match), or it was not (and recovery will detect the truncation).

Snapshots. If left unchecked, the WAL would grow without bound. We take a snapshot every N committed commands (N=1000 in our tests) capturing the block tree, committed log, `high_qc`, and eventually the application state. Once a snapshot succeeds, all WAL entries before its sequence number can be safely deleted. Snapshots are serialised with `bincode` via `serde`, which gives us compact binary output without the overhead of a text format.

Recovery. The procedure is straightforward (Algorithm 1): load the most recent snapshot, replay any WAL entries that came after it, verify each entry's checksum, and rejoin the protocol. If a checksum fails, the replica halts rather than risk operating on a corrupted state; this is a deliberate choice. In a Byzantine setting it is better to stop and ask for help than to silently diverge from the rest of the quorum.

4.2 Dynamic Membership

Static validator sets are a real operational headache. If a node dies permanently, you are stuck with reduced capacity. If you want to scale up, you need to restart everyone. We propose handling membership changes through the consensus protocol itself, which avoids introducing a separate coordination mechanism.

The idea is simple enough: a `JoinValidator` or `RemoveValidator` command is included in a block like any other client request. It flows through proposal,

Algorithm 1 Recovery from snapshot + WAL

Require: Snapshot store, WAL store
Ensure: Recovered replica state
1: *snap* ← LoadSnapshot()
2: *state* ← Restore(*snap*)
3: *entries* ← LoadWAL(*snap.seqNo*)
4: **for all** *entry* ∈ *entries* **do**
5: **if** VerifyCRC(*entry*) **then**
6: Apply(*state*, *entry*)
7: **else**
8: **halt** with error
9: **end if**
10: **end for**
11: **return** *state*

voting, and QC formation normally. The membership change only takes effect once the enclosing block is committed via the 3-chain rule. This gives us atomicity for free; either the change commits or it does not; and all honest replicas agree on which blocks were committed.

For joins, the new replica would need to catch up by downloading the latest snapshot and replaying subsequent WAL entries from an existing peer. For removals, the departing replica simply stops participating after the commit.

One subtlety we want to flag: quorum intersection across configurations. In a single configuration, any two quorums of size $2f+1$ out of $3f+1$ must overlap in at least one honest replica. When the configuration changes, we need to make sure decisions made under the old quorum are respected by the new one. Our approach is to apply the change atomically *after* commitment, so the finalised chain prefix is locked in under the old configuration's quorum rules before the new configuration activates. We formally verify this property and six additional membership safety invariants via exhaustive TLC model checking (see Section 5.3).

4.3 Pluggable State Machine

Most HotStuff prototypes just append commands to a log and call it a day. We want the consensus layer to be oblivious to what it is ordering. Whether it is a key-value store, a counter, a blockchain ledger, whatever the application needs.

Our design defines a Rust `App` trait with three methods: `apply(cmd)` to execute a command, `snapshot()` to capture application state, and `restore(state)` to rebuild it. This is directly inspired by Tendermint's ABCI [8]; we make no claim of novelty on the interface itself. The value is in integrating it cleanly with the WAL/snapshot machinery described above.

For applications that are not written in Rust, we plan to expose the same three operations over gRPC [9]: an `Execute` RPC replaces `apply`, and `Snapshot`/`Restore` RPCs handle state persistence/safety. We have also considered WASM

as an alternative for sandboxing untrusted plugins, though that remains speculative at this point.

4.4 Enhanced Pacemaker

The liveness problem under low load is subtle but annoying. Say a client submits one command. The leader wraps it in a block and gets a QC. However, the 3-chain rule means that the block will not commit until two more QC-justified blocks appear after it. If no more client commands arrive, those blocks never get proposed, and the original command sits in limbo indefinitely.

Our fix is for the pacemaker to monitor two conditions: (a) has it been longer than `timeout_ms` since the last proposal? and (b) are there fewer than three pending blocks? If both hold and the client queue is empty, the leader proposes a *dummy block*; a block with a no-op command. Dummies go through the full consensus path (proposal, vote, QC) and fill out the 3-chain. At the application layer, they are simply ignored.

Focus is wiring it into HotStuff's pacemaker correctly so that (a) dummies do not starve real commands, and (b) the commit latency for a single command is bounded by roughly $3 \times \mathit{timeout}$.

4.5 Consensus Core

The algorithm for the core consensus protocol was mentioned in Section 2. Given below are the core data structures used to implement it.

Blocks and the Block Tree. Each block is a tuple $B = \langle \mathit{hash}, \mathit{parent}, \mathit{view}, \mathit{proposer}, \mathit{qc}, \mathit{cmds} \rangle$. Blocks form a tree rooted at a genesis block; we store it as a `BTreeMap<Hash, Block>` for $O(\log n)$ lookups.

Cryptographic Authentication. We use Ed25519 [11] throughout. Each replica generates its own signing key at startup and publishes the corresponding verifying key to all peers. Votes are signed over $\texttt{SHA-256}(\mathit{block\_hash} \| \mathit{view})$, and a QC requires $\geq 2f+1$ valid signatures from distinct replicas. We chose Ed25519 over, say, Boneh–Lynn–Shacham (BLS) because it is fast, widely supported, and the `ed25519-dalek` crate is mature.

Voting and QC Formation. A replica votes for a proposal if: the proposer is the legitimate leader for that view, the parent block exists locally, and any embedded QC verifies. We also reject duplicate votes; same signer, same block. Once the leader accumulates $\geq 2f+1$ valid votes, it assembles a QC.

Commit Rule. Algorithm 2 formalises what happens: for every uncommitted block B_0, we look for a child B_1 and a grandchild B_2, both QC-justified. If the chain exists, B_0 commits. In practice, commits happen at the speed of proposal rounds.

Algorithm 2 3-chain commit detection

Require: Block tree $\mathcal{T}$, committed set $\mathcal{C}$
Ensure: Updated $\mathcal{C}$
for all $B_0 \in \mathcal{T} \setminus \mathcal{C}$ **do**
  $B_1 \leftarrow \textsc{Child}(\mathcal{T}, B_0)$
  if $B_1 = \perp$ **or** $B_1.qc = \perp$ **then**
    continue
  end if
  $B_2 \leftarrow \textsc{Child}(\mathcal{T}, B_1)$
  if $B_2 = \perp$ **or** $B_2.qc = \perp$ **then**
    continue
  end if
  $\mathcal{C} \leftarrow \mathcal{C} \cup \{B_0\}$
end for
return $\mathcal{C}$

Table 3. Simulation results (n=4, f=1, single-process)

Rounds	Proposed	Committed	Messages	Safety Viol.
100	100	97	800	0
500	500	495	4000	0
1000	1000	998	8000	0

Leader Rotation. Nothing fancy: $leader(v) = v \bmod n$. If a view times out, replicas increment their view number and move on. We may explore reputation-based rotation later, but round-robin is fine for now.

5 Evaluation

We evaluate PersistHotStuff on three axes: correctness under adversarial conditions (simulation and stress tests), formal safety verification of dynamic membership (TLA+ model checking), and comparative performance against PBFT and Raft.

5.1 Simulation Results

Table 3 summarises multi-replica simulations (n=4, f=1). The key finding is **zero safety violations** across all scales; the 2–5 block gap between proposed and committed counts reflects the trailing 3-chain.

5.2 Stress Testing

Crash recovery. We ran 120 consecutive kill/restart cycles (n=4, f=1), recovering each victim from its most recent snapshot plus WAL replay. All **120/120**

Table 4. Dynamic membership evaluation (35 scenarios, 0 violations)

Scenario	Transition	Runs	Pass
A – Valid join	$3 \to 4$	10	10
B – Valid remove	$5 \to 4$	10	10
C – Reject bad join	$4 \to 5$ (invalid)	5	5
D – Reject bad remove	$4 \to 3$ (invalid)	5	5
E – End-to-end cycle	3→4→5 (rejected)	5	5

recoveries succeeded with **zero safety violations**; 844 blocks committed (≈7.0 per cycle).

Dynamic membership. Table 4 summarises 35 membership-change scenarios. Valid transitions (3→4, 5→4) commit via the 3-chain rule; invalid transitions (4→5, 4→3) are rejected by the `can_apply_new_validator_count` invariant ($n \geq 4$, $n \bmod 3 = 1$). All 35/35 pass with zero violations.

Commit latency (pacemaker). Without the dummy-proposal pacemaker, 0/50 trials commit within 200 rounds (the 3-chain stalls). With the pacemaker, all 50/50 commit in exactly 3 rounds the theoretical minimum.

5.3 Formal Verification of Dynamic Membership Safety

We developed a TLA+ specification [12] that models the dynamic membership protocol. The specification captures the full consensus protocol extended with `JoinValidator` and `RemoveValidator` actions that modify the active validator set atomically at commit time.

Model parameters: N=4 replicas (bootstrap set $\{0, 1, 2\}$ with one candidate), fault bound f=0 for the bootstrap configuration, MAX_VIEW=4, MAX_HASH=9, MAX_EPOCH=2.

Safety invariants verified: We specify seven membership-specific invariants plus four baseline consensus invariants, all checked by TLC:
1 ValidatorSetBFT: $|V_r| \geq 3f{+}1$ post-activation.
2 QuorumIntersection: any two quorums of V_r overlap.
3 CommitSafetyAcrossEpochs: no cross-epoch prefix contradictions.
4 EpochMonotone: configuration epochs never decrease.
5 ValidTransitionsOnly: $n \geq 4 \wedge n \bmod 3 = 1$.
6 ConsistentMembershipView: same-epoch replicas agree on V.
7 QCsAlwaysValid: QC signers $\subseteq V$ at QC's epoch.
Four additional invariants (**NoDoubleVote**, **UniqueHashes**, **CommittedBlocksInTree**, **VotePoolSignersValid**) are also verified.

TLC results: Exhaustive model checking explored **7.3 M distinct states** (20.5M generated, depth 13) with **zero violations** across all 11 properties. The `CommitBlock` action fired 35 times, confirming that the model reaches committed membership transitions ($3 \to 4$ via `JoinValidator`).

Table 5. Message complexity per consensus round

n	f_{BFT}	**PBFT** $2n^2-n$	**Raft** $2(n-1)$	**PHS** (measured)
4	1	28	6	9
7	2	91	12	18
10	3	190	18	27
13	4	325	24	36

5.4 Performance Comparison

We compare PersistHotStuff's measured performance against analytical models of PBFT [2] ($O(n^2)$ messages) and Raft [3] ($O(n)$ messages, crash-fault only) at cluster sizes $n \in \{4, 7, 10, 13\}$ all valid under our $n \geq 4,\ n \bmod 3 = 1$ invariant. For each size we run 20 trials measuring message count, commit latency, and throughput.

Table 5 confirms that PersistHotStuff achieves $3(n-1)$ messages per round, exactly the analytical HotStuff bound (proposal + votes + QC broadcast). The persistence layer adds only local disk I/O, no extra network messages. At n=13, PBFT requires 9.0× more messages (325 vs. 36), while Raft uses fewer messages (24) but tolerates only crash faults, not Byzantine faults.

As n grows from 4 to 13, PBFT messages grow 11.6× (quadratic) while PersistHotStuff grows only 4.0× (linear), matching Raft's scaling with strictly stronger Byzantine fault tolerance.

Commit latency: PersistHotStuff commits every client command in exactly 3 rounds across all cluster sizes. This is the theoretical minimum imposed by the 3-chain rule. PBFT and Raft achieve single-round commits, but PersistHotStuff trades this small latency increase for its $O(n)$ message complexity advantage.

Throughput: Steady-state throughput is 1.0 commits/round at all cluster sizes, confirming that the protocol sustains pipeline-depth-1 throughput without degradation as n grows.

6 Discussion

6.1 Design Trade-offs

- **In-memory block tree.** We keep the entire block tree in RAM, which is fine for thousands of blocks but will not scale to millions. A natural extension is to page older blocks to disk, using the WAL infrastructure that already exists.
- **Consensus-driven membership.** Routing membership changes through the consensus log is simple but introduces latency(the change only takes effect after 3-chain commit). Systems that need rapid reconfiguration might prefer a separate fast path.

– **No threshold signatures.** QCs in our system carry $2f+1$ individual Ed25519 signatures, which grows linearly in f.Threshold (or aggregate) signatures could compress a QC into a single short proof but add cryptographic complexity. We defer this optimisation to future work.

6.2 Safety and Liveness

Safety. The consensus core follows HotStuff's 3-chain commit rule without modification, so the original safety proof [1] applies. The WAL and snapshots add durability but do not change when or how blocks are committed. Dummy proposals are regular blocks that carry no-op commands; they pass through the same voting and QC pipeline, so they cannot weaken safety. For dynamic membership, our TLA+ specification (Section 5.3) provides machine-checked verification.

Liveness. HotStuff guarantees liveness after Global Stabilisation Time(GST) assuming an honest leader. Our pacemaker extension adds liveness under low load by ensuring the 3-chain always has enough blocks to make progress. For dynamic membership, liveness depends on the new configuration having enough honest replicas, which is enforced by the $n' \geq 4 \wedge n' \bmod 3 = 1$ invariant.

7 Conclusion

We have presented the design and evaluation of PersistHotStuff, a modular framework that closes four practical gaps in existing HotStuff prototypes: crash-safe persistence, dynamic membership, pluggable application logic, and liveness under low load.

All four extensions are fully implemented and rigorously tested in Rust. Beyond simulation and stress testing, we provide two new contributions that strengthen the paper's technical foundation:

– A TLA+ specification of the dynamic membership protocol, verified by TLC across 7.3 M distinct states with zero violations of 11 safety invariants; including quorum intersection, commit safety across epochs, and valid transition enforcement.
– A performance benchmark at cluster sizes $n \in \{4, 7, 10, 13\}$ confirming $O(n)$ message complexity ($3(n-1)$ messages/round, matching the analytical HotStuff bound) and demonstrating 9.0× fewer messages than PBFT at n=13.

The implementation is available on GitHub [13]. If there is one takeaway, it is that the gap between a BFT *protocol* and a BFT *system* is larger than it looks. Storage, membership, application interfaces, and liveness under real workloads all matter, and addressing them together rather than one at a time forces design decisions that would not surface otherwise. We hope this blueprint is useful to others working in the same space.

References

1. Yin, M., Malkhi, D., Reiter, M.K., Gueta, G.G., Abraham, I.: HotStuff: BFT Consensus in the Lens of Blockchain. In: Proc. ACM PODC (2019). https://arxiv.org/abs/1803.05069
2. Castro, M., Liskov, B.: Practical Byzantine Fault Tolerance. In: Proc. 3rd USENIX OSDI, pp. 173–186 (1999). https://css.csail.mit.edu/6.824/2014/papers/castro-practicalbft.pdf
3. Ongaro, D., Ousterhout, J.K.: In Search of an Understandable Consensus Algorithm. In: Proc. USENIX ATC, pp. 305–320 (2014). https://raft.github.io/raft.pdf
4. Kang, D. et al.: HotStuff-1: Linear Consensus with One-Phase Speculation. arXiv:2408.04728 (2024). https://arxiv.org/pdf/2408.04728
5. Abraham, I., Malkhi, D., Nayak, K., Ren, L., Yin, M.: Sync HotStuff: Simple and Practical Synchronous State Machine Replication. In: Proc. IEEE S&P (2020) https://ieeexplore.ieee.org/document/9152792
6. Guo, H. et al.: DynaNet: A Dynamic BFT Consensus Framework. J. Systems Architecture (2025). https://www.sciencedirect.com/science/article/abs/pii/S138376212500058X
7. Stanford CS244B: HotStuff Implementation and Advice. Course Project Report, Spring 2024. https://www.scs.stanford.edu/24sp-cs244b/projects/HotStuff_Implementation_and_Advice.pdf
8. Kwon, J.: Tendermint: Consensus Without Mining (2014). https://tendermint.com/static/docs/tendermint.pdf
9. gRPC: A High Performance, Open Source Universal RPC Framework. https://grpc.io/
10. Dwork, C., Lynch, N., Stockmeyer, L.: Consensus in the Presence of Partial Synchrony. J. ACM **35**(2), 288–323 (1988) https://dl.acm.org/doi/epdf/10.1145/42282.42283
11. Brendel, J., Cremers, C., Jackson, D., Zhao, M.: The Provable Security of Ed25519: Theory and Practice (2021). https://ieeexplore.ieee.org/document/9519456
12. Chaudhuri, K., Doligez, D., Lamport, L., Merz, S.: A TLA+ Proof System. In: CEUR Workshop Proceedings (2008). https://www.researchgate.net/publication/220896380_A_TLA_proof_system
13. Shet, S., Kolhe, T.: PersistHotStuff-Rust. https://github.com/SaanviShet/persisthotstuff-rst